Drop The Billionaire API

Oct 27, 2026 · @lam

The game has three POST endpoints under `/api/dropTheBillionaire`, all sent with `gameId` = `dropthebillionaire` and every body value Base64-encoded.

Calling the API

All endpoints are `POST` with a JSON body, on the same HashBet game server and path pattern as the other games (for example `/api/flappyBird`): `<game server host>/api/dropTheBillionaire/<endpoint>`.

Every value in the request body must be Base64-encoded, numbers included (encode the number as text). The server decodes each value before validating it.

Field	Plain value	Sent as
`gameId`	`dropthebillionaire`	`ZHJvcHRoZWJpbGxpb25haXJl`
`token`	the player's session token	Base64 of the token
`totalBet`	`2`	`Mg==`

```
// Leaves out fields that are undefined or null, so they are never sent as
the text "undefined".
const encode = (body) =>
  Object.fromEntries(Object.entries(body)
    .filter(([, value]) => value !== undefined && value !== null)
    .map(([key, value]) => [key, btoa(String(value))]));

const res = await fetch(`${host}/api/dropTheBillionaire/betPlace`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(encode({ gameId: "dropthebillionaire", token,
totalBet: 2 }))
});
if (!res.ok) {
  /* 429: wait and retry; anything else: show a generic error */
} else {
  const data = await res.json();
  if (data.errorCode) { /* show the error, see Errors */ }
}
```

Game replies are HTTP 200 JSON; a failure carries `errorCode` (see Errors). Check `res.ok` before `res.json()`, because a few replies are not JSON:

- HTTP 429: more than 300 requests in a minute with one token, counted across all endpoints (300 is the server's default setting). The body is the plain text `Too many requests, please try again later.`
- HTTP 400 (malformed JSON), 404 (wrong path), 413 (body over 16 KB) or 500 (the server's rate-limit store is down): HTML body.

Call order:

1. `init` when the game loads: the player's balance and their last round.
2. `betPlace` once per round: one call returns the whole finished round (road, landing, win, new balance). Play the animation from it; there is no cash-out or step call.
3. `getPreviousRounds` for the bet history panel.

POST /init

Call once when the game loads to get the balance and the player's last round.

Field	Required	Value
`gameId`	yes	`dropthebillionaire`
`token`	yes	the player's session token

Response shape (`playerInfo` values shortened):

```
{
  "response": { "gameAlias": "dropthebillionaire", "...": "the player's last-round record from the RGS" },
  "playerInfo": { "externalPlayerId": "p1", "balance": 100, "currency": "USD"
}
}
```

- `playerInfo` is the RGS player info (balance, currency and so on) with `gameMode` and `PPR` removed, the same as in the other HashBet games.
- `response` is the RGS's last-round record, passed on with seeds, `gameMode` , `pprMultiplier` and money fields removed. A player with no rounds gets `{ "gameAlias": "dropthebillionaire" }` .
- Every round finishes inside `betPlace` , so there is never an open round to resume; `init` is mainly for the balance.
- Open question: where the last round's result sits in `response` is not yet confirmed against a real staging response. In the other HashBet games the RGS keeps it under `response.extraData` . Until confirmed, show a last result only if you find a `road` array there.

POST /betPlace

Places the bet and returns the whole round, already settled: the balance in the reply already includes the win.

Field	Required	Value
`gameId`	yes	`dropthebillionaire`
`token`	yes	the player's session token
`totalBet`	yes	bet amount, a positive number (for example `2`)

Response (a real round at bet 2, balance 100 before the round):

```
{
  "gameId": "dropthebillionaire",
  "playerId": "player1",
  "roundId": "01a17b1e-ccac-7512-8d4a-ff7bc03e0afb",
  "closed": true,
  "result": {
    "gameAlias": "dropthebillionaire",
    "totalBet": 2,
    "spinResult": {
      "bet": 2,
      "multiplier": 1.2,
      "totalWin": 2.4,
      "closed": true,
      "maxPayout": 100000,
      "isWin": true,
      "road": ["+0,2", "+0,2", "1.5", "1.5", "+0,3"],
      "finalZone": "Grass",
      "finalZoneCode": 1
    },
    "validStake": { "total": 2 }
  },
  "balance": 100.4
}
```

`spinResult` field	Type	Meaning
`multiplier`	number	The win as times the bet, 2 decimals. `0` = lost. Show it as `multiplier.toFixed(2) + "x"`.

`spinResult` field	Type	Meaning
`totalWin`	number	Money paid to the player: `bet` × `multiplier`.
`isWin`	boolean	`true` whenever `multiplier` > 0, including wins below 1x.
`road`	string[]	What the billionaire hits, in order. See Reading the road.
`holeInfo`	object, only when a black hole appears	`blackHoleBonus`: amount the black hole adds to the win counter, at bet 1 (3.00 to 5.99). `index`: position of `"hole"` in `road`.
`finalZone`	string	Landing zone name.
`finalZoneCode`	number	Landing zone multiplier.
`deathType`	string, only on an instant death	`"Eagle"` or `"Heli"`.
`bet`	number	The bet.
`closed`	boolean	Always `true`: the round is finished.
`maxPayout`	number	The RGS max payout, as money (not times the bet). A round also never pays more than 600x the bet, so the largest possible win is the smaller of 600 × bet and maxPayout.

The top-level `balance` is the balance after the round with the win already added (100 − 2 bet + 2.4 win = 100.4): show it as it is and do not add `totalWin` again. `result.validStake` is for the RGS and can be ignored. `roundId` identifies the round in the bet history.

Reading the road

Start the win counter at 0 and apply each `road` item in order; after the last one, multiply by `finalZoneCode` to get `multiplier`. Road values are at bet 1, so multiply by the bet to show money.

Token	Item	Effect on the win counter
`+0,2`	Cap	+ 0.2
`+0,3`	Burger	+ 0.3
`+0,4`	Coin	+ 0.4
`+0,5` `+1` `+2` `+5`	Money cloud	+ the value (the comma is the decimal point)
`1.5` `2` `3` `5`	Multiplier	× the value
`/2`	Divide	÷ 2
`hole`	Black hole	+ `holeInfo.blackHoleBonus`

`finalZone`	`finalZoneCode`	Landing
`Grass`	1	Normal landing; also sent on an instant death
`Nobel`	4	Nobel Prize, ×4
`Oil`	8	Oil Rig, ×8
`Greenland`	15	Greenland, ×15
`WhiteHouse`	20	White House, ×20

- A road has 5 to 16 items: 1 to 3 starting items, then 4 to 12 falling items, plus at most one `hole`. Nothing marks where the starting items end, as in the reference game.
- **Instant death**: `road` is `[]`, `multiplier` is `0`, `deathType` is `"Eagle"` or `"Heli"`, and `finalZone` is `Grass`.
- Use the server's `multiplier` and `totalWin` for the final numbers. Replaying the road in your own code is only for the animation; floating-point maths can differ in the last digit.

Worked example, the black-hole round `["+0,2","+0,4","+0,2","/2","+0,4","2","/2","/2","+0,2","hole","/2","/2"]` with `blackHoleBonus` 3.17 on `Grass`: 0.2 + 0.4 + 0.2 = 0.8 → 0.4 → 0.8 → 1.6 → 0.8 → 0.4 → 0.6 → black hole 3.77 → 1.885 → 0.9425 → ×1 = `multiplier` 0.94 (rounded half up to 2 decimals).

```
// Win counter (at bet 1) after each road item, for the animation.
function replayRoad(spin) {
  let counter = 0;
  return spin.road.map((token) => {
    if (token === "hole") counter += spin.holeInfo.blackHoleBonus;
    else if (token === "/2") counter /= 2;
    else if (token.startsWith("+")) counter +=
Number(token.slice(1).replace(",", "."));
    else counter *= Number(token);
    return counter;
  });
}
```

POST /getPreviousRounds

Without `roundId` it returns the player's bet history list; with `roundId` it returns that one round's detail.

Field	Required	Value
`gameId`	yes	`dropthebillionaire`
`token`	yes	the player's session token
`roundId`	no	a round's id, for its detail
`pageNo`	no	page number, 1 or more (default 1)
`pageSize`	no	rows per page, 1 to 100 (default 10)

Send no other fields: an extra field such as `gameType` is rejected with error 2001.

- **History list** (no `roundId`): the RGS bet history, passed through unchanged, the same as in the other HashBet games.
- **One round** (`roundId` set): the round's stored result under `detailHistory.extraData`, with the same `spinResult` fields as `betPlace`. Seeds are not returned for this game.

```
{
  "result": true,
  "roundId": "0192f0c3-7a10-7c3e-9d2a-5b1e2f3a4b5c",
  "detailHistory": {
    "roundId": "0192f0c3-7a10-7c3e-9d2a-5b1e2f3a4b5c",
    "extraData": [
      {
        "gameAlias": "dropthebillionaire",
        "totalBet": 2,
        "spinResult": {
          "bet": 2, "multiplier": 0.94, "totalWin": 1.88, "closed": true,
"maxPayout": 100000, "isWin": true,
          "road":
["+0,2","+0,4","+0,2","/2","+0,4","2","/2","/2","+0,2","hole","/2","/2"],
          "finalZone": "Grass", "finalZoneCode": 1,
          "holeInfo": { "blackHoleBonus": 3.17, "index": 9 }
        }
      }
    ]
  }
}
```


Open question: the RGS sets the wrapper around `extraData`, and this shape is not yet confirmed against a real staging response. In the other HashBet games the stored round sits under `extraData`, as shown; read `spinResult` defensively until confirmed.

Errors

A failed call still returns HTTP 200, with `errorCode` and a readable `error` (server and RGS errors also add `detailedMsg`). Check for `errorCode` before reading the result.


```
{ "errorCode": 1002, "error": "Bet Place Validation error: \"token\" is
required" }
```


`errorCode`	When
1001	`gameId` not in the body (an empty value gives 2001)
1002	`token` not in the body (an empty value gives 2001)

`errorCode`	When
1008	`totalBet` not in the body (`betPlace`)
1015	A body value could not be Base64-decoded
2001	Any other bad request: wrong `gameId`, `totalBet` not a positive number, an empty value, or an extra field
1101	`betPlace` failed on the server after the bet was taken; no win is paid
1103	The RGS sent no valid max payout; the bet was already taken and no win is paid
1999	`init` failed unexpectedly
1401	`getPreviousRounds` failed
500	Internal server error
any other code	From the RGS, usually with its own `error` and `detailedMsg` (for example insufficient balance or an expired token). If the RGS refuses the payout, `error` is `Internal Server Error` and `detailedMsg` is `Error when crediting player`.

Choose the player's message by `errorCode`, not by the `error` text. After a `betPlace` error other than a request error (1001, 1002, 1008, 1015, 2001) or HTTP 429, the bet may already have been taken: call `init` to reload the balance instead of restoring the old one.